DAY 2 · FRIDAY 30 OCTOBER 2026

In the cloud

Yesterday on your laptop. Today: deploy it, automate it, run it.

code ▷ container ▷ cloud ▷ automation ▷ operations

David Cediel

Yesterday in one slide

Landscape

IaaS → CaaS → PaaS → FaaS: you do less, control less

Containers

An image packs the app and everything it needs. Config from env vars.

Compose

Many containers, one file. Data lives in volumes, not containers.

Kubernetes

You declare, it keeps it true. Pod, Deployment, Service, Ingress, HPA.

Today: **your image from yesterday goes to the cloud**, then we automate it and look at production.

SETUP CHECK · 2 MIN

Ready for the cloud?

1 · YOUR IMAGE

The api image builds and runs (yesterday's homework)

2 · DOCKER HUB

An account, and you know your username

3 · AZURE

portal.azure.com open, Azure for Students active

No image? Use the ready one in Hands-on 2: davidcediel12/seminar-api:1.0

BLOCK 3B

Running services in the cloud: practice

Goal: your API on a public URL.

- How a release goes live
- Demo: Container Apps
- Demo: Online Boutique
- Hands-on 2

Releasing a new version: four strategies

Recreate

– – – –

Stop all old, then start all new

+ simple

– downtime

Rolling

v2 v2 v1 v1

Replace copies one at a time (the Kubernetes default)

+ no downtime, no extra servers

– two versions live for a while

Blue/green

v1 v1 v2 v2

Two full copies; switch all traffic at once

+ instant switch and rollback

– double the resources

Canary

v1 v1 v1 v2

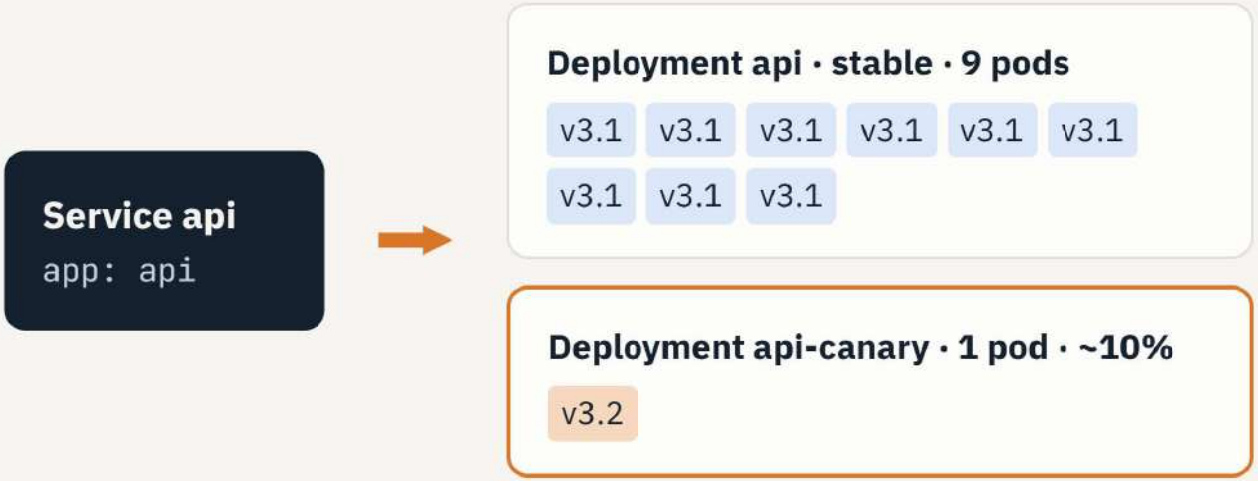
Send 5% of users to v2, watch, then grow

+ a bug hits few users

– needs traffic splitting and monitoring

Rollback = deploy the previous version again. Easy only if every image has its own version tag, never just :latest.

Canary in plain Kubernetes



1 Add the canary Deployment, 1 pod

2 Compare RED: canary vs stable

3 Good: shift pod counts, repeat

4 Done: new tag in api, delete the canary

Bad: delete the canary, all back on v3.1

Kubernetes has no canary feature: you do it by hand. On Container Apps it’s built in, with revision weights (90/10).

Health checks: how Kubernetes knows

LIVENESS · /livez

Is it alive?

No answer 3 times in a row: Kubernetes **restarts** the container.

READINESS · /readyz

Can it take traffic?

No: the Service **stops sending requests** to it, without restarting it.

Our API answers /readyz with 503 while it shuts down, so no request lands on a pod that is leaving.

A rolling update without downtime

1. Start **one** new pod (v2)
2. Wait until **/readyz** says it is ready
3. Stop **one** old pod; it finishes its requests first
4. Repeat until all pods run v2

FAILED REQUESTS DURING THE UPDATE

0

Tested with our API: requests kept coming during the whole update, and none failed.

```
maxSurge: 1 · maxUnavailable: 0
```

Container Apps: Kubernetes, managed for you

YOU GIVE

An image, env vars, secrets, scaling rules

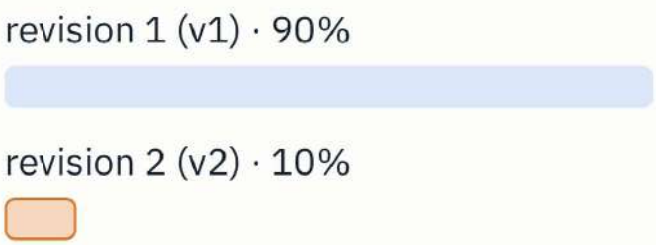
CONTAINER APPS GIVES

An HTTPS URL, revisions, traffic splitting, scale to zero

UNDERNEATH, HIDDEN

A Kubernetes cluster that Microsoft runs for you

A CANARY, BUILT IN



Every change creates a new revision; you choose how traffic is split.

Same idea elsewhere: Azure Container Apps · AWS ECS Express Mode (Fargate) · Google Cloud Run

Today, Hands-on 2: your image from yesterday, running here on a public URL.

LIVE DEMO · 4 MIN

Our API on Container Apps

1. Push the image to **Docker Hub**
2. Portal: **Container Apps** → **Create**, point it to the image
3. Open the public **HTTPS URL**
4. Change **APP_MESSAGE**: a new revision, no rebuild

WATCH FOR

HTTPS and a URL, with no setup

Env vars and secrets in the portal

Revisions, like yesterday's Deployment

The first call is slow: scale from zero

LIVE DEMO · 3 MIN first to cut if late

Online Boutique: the same ideas at scale

Google’s demo web shop, running on **AKS**.

11 services in different languages, talking to each other.

One YAML file describes the whole system.

Our app has 3 services. Same concepts, bigger scale.

Inspiration only: **not** part of the assignment.

```
# everything running
kubectl get pods
# self-healing: delete one, watch it return
kubectl delete pod -l app=cartservice
kubectl get pods -w
# scaling in one line
kubectl scale deployment frontend --replicas=3
```

HANDS-ON 2 · 20 MIN README section 5 ⚠ Build on **step-1-containerized** (or your step-0), not step-2: it needs a database

Deploy your API to Container Apps

1 · Push
5 min

```
docker login
docker build -t <you>/seminar-api:1.0 ./api
docker push <you>/seminar-api:1.0
```

2 · Create
10 min

Portal: **Container Apps** → **Create**. The fields that matter are on the next slide.

3 · Open
5 min

Overview → **Application Url**, add **/api/message**. Your API, on public HTTPS.

No image? Use davidcediel12/seminar-api:1.0 · **Done?** Add APP_MESSAGE → a new revision

HANDS-ON 2 · STEP 2

The fields that matter in the portal

Tab	Set this
Basics	New resource group rg-seminar, name seminar-api, a region, new environment
Container	Uncheck “Use quickstart image”. Docker Hub or other registries, Public, docker.io, <you>/seminar-api:1.0
Ingress	Enabled, traffic from anywhere, target port 8080
Then	Review + create, wait about a minute

Port **8080** = the port inside the container. Azure maps public HTTPS to it, like -p did locally.

QUIZ · BLOCK 3B · 1 OF 2

Which release strategy lets you test a new version with 5% of users?

- **A** Recreate
- **B** Rolling update
- **C** Blue/green
- **D** Canary

Answer in the chat: **A, B, C or D**

ANSWER · BLOCK 3B · 1 OF 2

Which release strategy lets you test a new version with 5% of users?

A Recreate

B Rolling update

C Blue/green

D **Canary** ✓

Why: A canary sends a small share of real traffic to the new version, compares its metrics with the old one, then widens.

QUIZ · BLOCK 3B · 2 OF 2

Why was the first request to your app slow?

- **A** Scale to zero: no copy was running (cold start)
- **B** The image is too big
- **C** Docker Hub was slow
- **D** HTTPS needs to warm up

Answer in the chat: **A, B, C or D**

ANSWER · BLOCK 3B · 2 OF 2

Why was the first request to your app slow?

A Scale to zero: no copy was running (cold start) ✓

B The image is too big

C Docker Hub was slow

D HTTPS needs to warm up

Why: With no traffic, Container Apps scaled to zero. The first request had to wait for a copy to start.

BLOCK 4

Automation: CI/CD and infrastructure as code

Goal: see why clicking in the portal doesn't scale.

- CI/CD
- Demo: GitHub Actions
- Infrastructure as code
- Demo: Terraform

You just deployed by clicking. Now do it for dev, test and prod, every day, and again after someone deletes something.

- Not repeatable
- Not reviewable
- Easy to get wrong
- Doesn’t scale

CI, CD and CD

CI

Continuous Integration

Every push is built and tested automatically. Small changes, merged often.

CD

Continuous Delivery

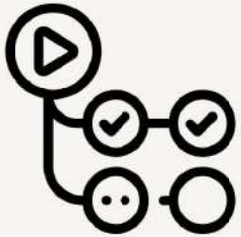
Every change that passes is ready to deploy. Production is one button.

CD · OUR REPO

Continuous Deployment

Every change that passes goes to production on its own.

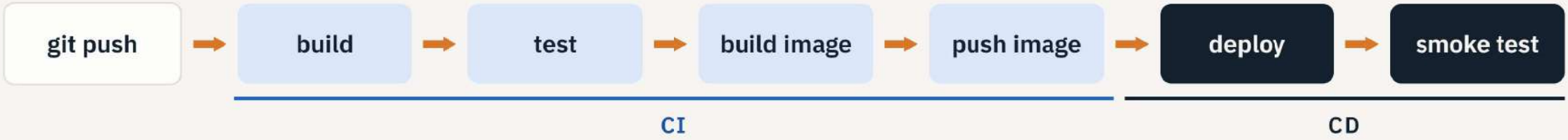
The difference between the two CDs: **who presses the button**, a person or the pipeline.






A pipeline, step by step



Build once, deploy many

The same image goes to every environment. Only the config changes.

Tag with the commit

seminar-api:3f2a1c9: every deploy traces back to its code.

Tools: GitHub Actions · ours · GitLab CI/CD · Azure Pipelines (Azure DevOps) · Jenkins · AWS CodePipeline · Google Cloud Build

Our pipeline in GitHub Actions

```yaml
# .github/workflows/deploy-api.yml (shortened)
on:
  push: { branches: [main], paths: ["api/**"] }
  pull_request:  # build only
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: docker/login-action@v4  # secrets.DOCKERHUB_TOKEN
      - uses: docker/build-push-action@v7
        with: { tags: "seminar-api:${{ github.sha }}" }
  deploy:
    needs: build
    steps:
      - uses: azure/login@v3  # OIDC, no password
      - run: az containerapp update --image …:${{ github.sha }}
```

on when it runs

jobs each on a fresh VM (a runner)

uses a ready-made action

run a shell command

needs deploy only if build passed

secrets encrypted, hidden in logs

github.sha the commit id

How the pipeline logs in to Azure, with no password

</>

GitHub Actions run

gets a signed token: “I am repo X, branch main”

Azure identity

trusts exactly that repo and branch, swaps it for a short-lived token

Container App

updated, with rights only on two resource groups

OLD WAY

A password stored in GitHub. It can leak and must be rotated.

OIDC · OUR REPO

Nothing secret stored. Tokens expire in minutes.

LIVE DEMO · 4 MIN branch **main**

From git push to live, with no clicks

1. Push a small change to the API
2. **Actions** tab: build job, then deploy job
3. Image on Docker Hub, tagged with the commit
4. **/api/message** shows the new version

WATCH FOR

The same steps as Hands-on 2, done by a machine

version in the response = the commit SHA

A failed build means nothing is deployed

Infrastructure as code

IMPERATIVE · THE STEPS

```
az group create -n rg-web …
az storage account create \
  -n stseminarweb123 -g rg-web
az storage blob service-properties \
  update --static-website …
```

Run it twice: **errors**, it already exists.

DECLARATIVE · THE END STATE

```
resource "azurerm_storage_account" "web" {
  name          = "stseminarweb…"
  account_tier = "Standard"
  account_replication_type = "LRS"
}
```

Run it twice: **“No changes”**.

Tools: Terraform / OpenTofu · any cloud | Bicep · Azure | CloudFormation · AWS | Pulumi · real code

Terraform in five words and four commands



Provider a plugin for one cloud's API (azurerm, aws, google)

Resource one thing to create, e.g. a storage account

Variable / output inputs, and values it prints at the end

State its record of what exists. Has secrets: **never commit it**

Drift someone changed it by hand; plan shows it

`init`	download providers
`plan`	preview: + create, ~ update, – delete
`apply`	make it so
`destroy`	delete everything it manages

Our repo, **step-4-terraform**: the frontend's storage account, created in about 30 seconds.

LIVE DEMO · 3 MIN branch **step-4-terraform** · infra/terraform

Change infrastructure by editing a file

```
terraform plan
# No changes. Matches the configuration.
# add to the storage account in main.tf:
tags = { course = "cloud-seminar" }
terraform plan
# ~ update in-place
terraform apply
```

WATCH FOR

Running it twice is safe

The plan says exactly what will change

The tag appears in the portal

QUIZ · BLOCK 4 · 1 OF 2

Why tag images with the commit SHA instead of :latest?

A It makes images smaller

B You know exactly which code runs, and rollback is easy

C Docker Hub requires it

D Builds get faster

Answer in the chat: **A, B, C or D**

ANSWER · BLOCK 4 · 1 OF 2

Why tag images with the commit SHA instead of :latest?

A It makes images smaller

B You know exactly which code runs, and rollback is easy ✓

C Docker Hub requires it

D Builds get faster

Why: A SHA points to exactly one commit: you know what runs, and rollback means redeploying an older tag. :latest keeps moving.

QUIZ · BLOCK 4 · 2 OF 2

What does terraform plan show you?

A What your resources cost

B The logs of the last apply

C The contents of the state file

D What will be created, changed or deleted, before anything happens

Answer in the chat: **A, B, C or D**

ANSWER · BLOCK 4 · 2 OF 2

What does terraform plan show you?

A What your resources cost

B The logs of the last apply

C The contents of the state file

D What will be created, changed or deleted, before anything happens ✓

Why: plan compares your files with what exists and lists + create, ~ update, – delete. Nothing changes until apply.

BLOCK 5

Operating in production

Goal: deploying is the beginning, not the end.

- **Secrets**
- **Observability**
- **Scaling, cost, security**
- **An incident**

Where does the database password go?

NEVER	**In the code or the image**	Stays in Git history and in every image layer
NEVER	**A committed .env file**	Fine locally, if it is in .gitignore
GOOD	**Env vars or secrets in the platform**	Container Apps secret, Kubernetes Secret. Enough for the assignment
BEST	**A secret store**	Key Vault, AWS Secrets Manager, Vault: audited, rotatable

A secret leaked to Git? **Rotate it.** Deleting the commit is not enough.

Observability: is it working, and why not?

Logs

What happened?

```
ERROR DB_URL is required
```

Metrics

How much, how often?

Requests per second,
errors, latency

Traces

Where is it slow?

POST /api/notes → SQL INSERT

OpenTelemetry is the open standard to collect all three, for any language and any tool.

One open-source stack: LGTM

L

Loki

stores logs

G

Grafana

dashboards over all of it

T

Tempo

stores traces

M

Mimir

stores metrics (or Prometheus)

app + OpenTelemetry → **collector** → **Loki · Tempo · Mimir** → **Grafana**

Same job, managed: Azure Monitor · AWS CloudWatch · Google Cloud Observability · Grafana Cloud · Datadog



Something is wrong: check RED first

R

Rate

Requests per second. A sudden drop: users can’t reach you.

E

Errors

Failed requests (5xx). A rising share: something broke.

D

Duration

How long requests take. Look at p95, not the average.

RED: which service hurts? → **Logs: what happened?** → **Traces: where exactly?**

LIVE DEMO · 3 MIN branch **extra-observability** · runs on my laptop

Our API, seen from the inside

1. Add a few notes in the app
2. Grafana dashboard: **RED** (rate, errors, duration) and notes created
3. One **log line** → its **trace**
4. Inside the trace: the request and its **SQL query**

WATCH FOR

Logs, metrics and traces of the same request, linked

Most of it is automatic, from a few lines of setup

No cloud needed

Scaling: bigger or more?

VERTICAL · SCALE UP

A bigger machine

Simple, but it has a ceiling and usually a restart.

HORIZONTAL · SCALE OUT

More copies

Almost no limit, survives failures. Needs a stateless app.

Autoscaling: our HPA went 2 → 4 → 2 pods on its own.

Scale to zero: no cost when idle, a cold start on the next call.

The usual bottleneck: the database, not the API.

Cost and security basics

COST

You pay for what **runs**, even if nobody uses it

A budget alert **warns**, it doesn't stop spending

One resource group per project, delete it at the end

Seen while preparing: a stopped Azure Postgres starts again by itself after 7 days

SECURITY

Least privilege: our pipeline can touch 2 resource groups, nothing else

The database is never public

HTTPS everywhere, no root in containers

Small images, scanned for known holes

DISCUSSION · A REAL INCIDENT

2012: one forgotten server, $460 million lost in 45 minutes

WHAT HAPPENED

Knight Capital, a big US trading firm, deployed new code to 8 servers **by hand**. One was missed and kept old code.

A reused config flag woke that old code up. It traded nonstop until people pulled the plug.

YOUR TURN

Which tools from these two days would have stopped it?

Fri 14:45 → 14:47

What would have saved Knight Capital

What went wrong	What prevents it	Where we saw it
Code copied by hand, server by server	A pipeline deploys, not a person	Block 4, GitHub Actions
One server ran different code	One image, identical copies everywhere	Blocks 2–3
Nobody knew which version ran where	Tag = commit SHA, version visible in the app	/api/message
Old code left behind, a reused flag	Immutable images: a new version replaces the old one	Rolling update
Error emails ignored, no automatic stop	Monitoring, alerts on errors, a kill switch	RED, Grafana
Rolled back the code, not the flag	Code and config released together; never reuse a flag	Revisions: image + env vars

Automated canary: release, watch, roll back

1 · RELEASE
5% of traffic to the new version

→

2 · A TOOL COMPARES
RED of new vs old, every minute

→

Healthy: 25% → 50% → 100%

Worse: automatic rollback to 0%

Tools: Argo Rollouts · Flagger reading metrics from Prometheus, the M in LGTM

KNIGHT CAPITAL, AGAIN

Orders per second jumped within seconds. An automatic check would have stopped it in about a minute, not 45.

WRAP-UP

What’s next for you

The journey, the assignment, and where to go from here.

- The journey
- The assignment
- Where to go next
- Questions

Recap

code	container	cloud	automation	operations
A small API, a frontend, a database	Dockerfile, image, Compose	Kubernetes, Container Apps, a public URL	GitHub Actions, Terraform	Secrets, RED, incidents

Every step is in the repo, one branch each: **step-0** to **step-5**, plus **extra-observability**.

ASSIGNMENT · DUE SUN 29 NOV, 23:59 SPAIN TIME

Your own 3-service app, in the cloud

Frontend
Yours, or our reference one
Blob Storage static website

→

API
Any language except Go, in a container
Container Apps

→

A managed resource
Your choice, really used by the API
Postgres, Redis, Cosmos DB…

Your own repo, not a fork of ours

App code is not graded: a tiny CRUD API is enough, AI-written is fine

Any cloud, but outside Azure it's on you

What you hand in

DELIVERABLES

Link to your Git repo, with your Dockerfiles

Screenshots of your cloud resources

A link to your video

THE VIDEO · 2–3 MIN · REQUIRED

The app working end to end, on its public URL

A walk through your resources in the cloud console

One problem you hit, and how you solved it

What you would change for real production

Send it to esdago18@gmail.com · **subject:** Cloud Seminar Assignment: <Name>

No secrets in code: connection strings go in Container App secrets. **Then delete your resources:** the video and screenshots are the proof.

Grading

Final grade: **15%** attendance + **85%** assignment

ASSIGNMENT · BASE UP TO 9/10

50%	The app works and the services talk to each other
25%	Good practices: env vars, no secrets in code, reasonable images
25%	The video: explanation and reflection

EXTRAS · ASSIGNMENT TO 10/10

+0.5 a CI pipeline (block 4)

+0.5 Terraform for your resource

+1 your API on Kubernetes (AKS, EKS, GKE)

Any combination, capped at 10.

Where to go from here

Not covered in these two days, but worth looking up next:

GitOps

Argo CD, Flux: the cluster pulls its state from Git

Helm

Packages of Kubernetes YAML, with templates

Service meshes

Istio, Linkerd: traffic and security between services

Certifications

Topic	Start here	Next step
Kubernetes	KCNA (Kubernetes and Cloud Native Associate)	CKAD (developer), CKA (administrator)
Azure	AZ-900 (Fundamentals)	AZ-204 (developer), AZ-104 (administrator)
AWS	Cloud Practitioner	Solutions Architect Associate
Google Cloud	Cloud Digital Leader	Associate Cloud Engineer
Terraform	Terraform Associate	

CKA and CKAD are hands-on exams in a real cluster: the closest to what we did here.

Thank you. Questions?

Repo: github.com/davidcediel12/cloud-deployments-seminar

Every command is in the README.

Contact and submissions: esdago18@gmail.com